

Program Designing 'and' { [Programming].ICT 1305

< J.M.D.S.Jayasinghe >

< Department of ICT >

< Faculty of Technology >

< Rajarata University of Sri Lanka >

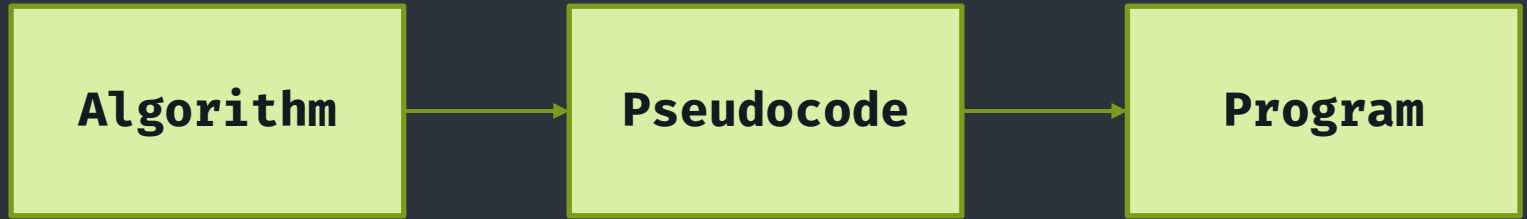
}

{ Pseudocode }

What is a Pseudocode {

- Flowcharts were the first design tool to be widely used, but unfortunately, they do not reflect some of the concepts of structured programming very well.
- Pseudocode, on the other hand, is a newer tool and has features that make it more reflective of the structured concepts.
- Pseudocode is a technique used to describe the distinct steps of an algorithm in a manner that's easy to understand for anyone with basic programming knowledge.

1
2
3
4
5
6
7
8
9
10
11
12
13
14



What is a Pseudocode {

- Pseudocode is representing 6 programming constructs:

- SEQUENCE
- CASE
- WHILE
- REPEAT-UNTIL
- FOR
- IF-THEN-ELSE

- These constructs are used to describe the control flow of the algorithm.

}

PSEUDOCODE CONSTRUCTS

SEQUENCE

Input: READ, OBTAIN, GET

Output: PRINT, DISPLAY, SHOW

Compute: COMPUTE,
CALCULATE, DETERMINE

Initialize: SET, INIT

Add: INCREMENT, BUMP

Sub: DECREMENT

FOR

FOR iteration bounds
sequence

ENDFOR

WHILE

WHILE condition
sequence

ENDWHILE

CASE

CASE expression OF
condition 1: sequence 1
condition 2: sequence 2

...

condition n: sequence n

OTHERS:

default sequence

ENDCASE

REPEAT-UNTIL

REPEAT
sequence

UNTIL condition

IF-THEN-ELSE

IF condition THEN
sequence 1

ELSE
sequence 2

ENDIF

The Structure Theorem {

- It is possible to write any computer program by using only three basic control structures that are easily represented in pseudocode:

- Sequence
- Selection
- Repetition

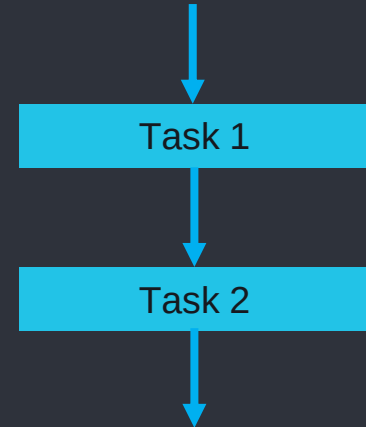
}

Sequence {

- Is the straight-forward execution of one processing step after another.

```
ADD 1 to pageCount  
PRINT heading line 1  
PRINT heading line 2  
SET lineCount to zero  
READ customer record
```

```
}
```

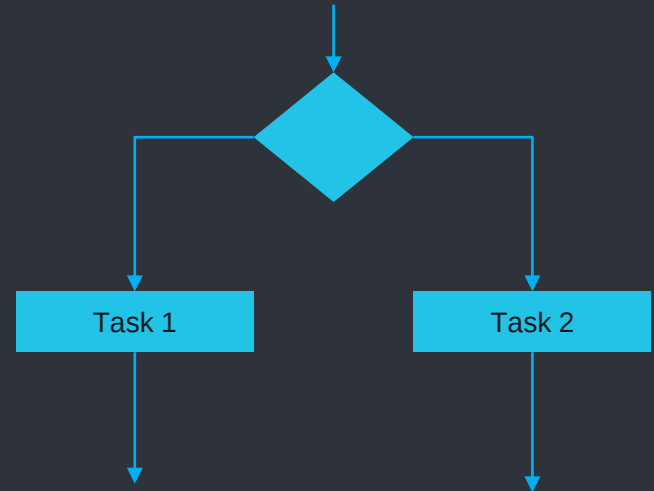


Selection {

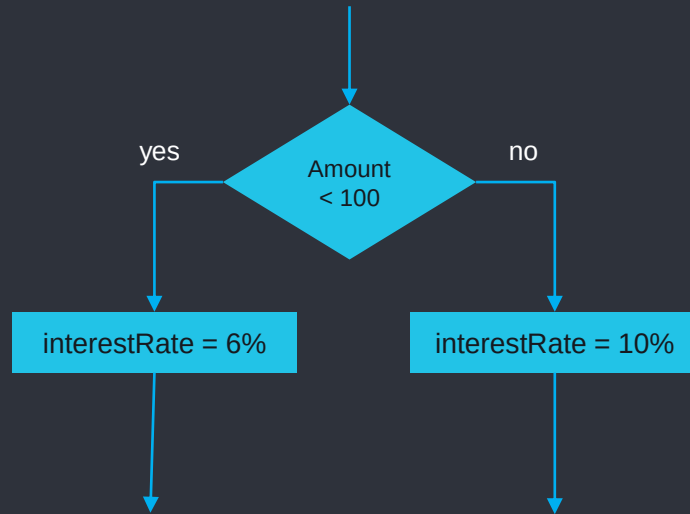
- Presentation of condition and the choice between two actions.

```
IF condition P is true THEN
    statement(s) in true case
ELSE
    statement(s) in true case
END-IF
```

```
}
```



Selection {



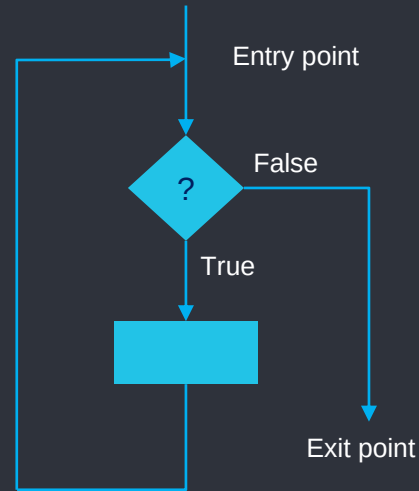
```
IF amount < 100 THEN
    interestRate = 6%
ELSE
    interestRate = 10%
END-IF
```

}

Repetition {

- The presentation of a set of instructions to be performed repeatedly, as long as a condition is true

```
WHILE condition P is true  
    statement(s) to execute  
END-WHILE
```



Repetition {

- Exaple:

```
SET student_total to 0
WHILE student_total < 50
    READ student record
    PRINT student name and address
    ADD 1 to student_total
END-WHILE
```

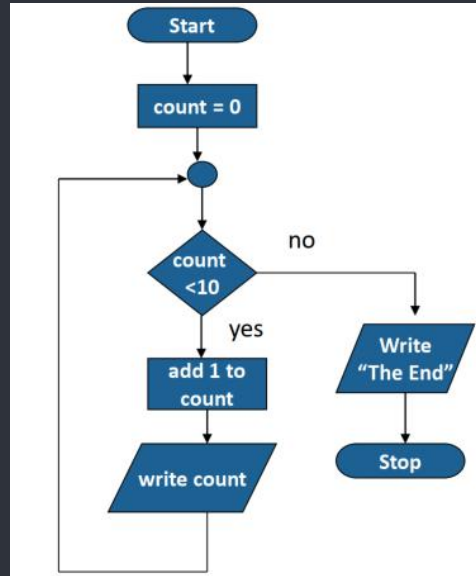
}

The Looping Structure {

- In flowcharting one of the more confusing things is to separate selection from looping.
- This is because each structure use the diamond as their control symbol.
- In pseudocode avoid this by using specific keywords to designate looping,
 - WHILE/ENDWHILE
 - REPEAT/UNTIL

}

WHILE / ENDWHILE {



```
count = 0
```

```
WHILE count < 10
```

```
    ADD 1 to count
```

```
    WRITE count
```

```
END-WHILE
```

```
WRITE "The end"
```

Mainline

```
count = 0
```

```
WHILE count < 10
```

```
    DO Process
```

```
END-WHILE
```

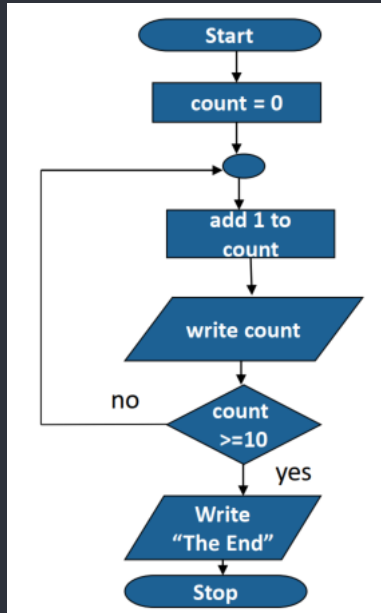
```
WRITE "The end"
```

Process

```
ADD 1 to count
```

```
WRITE count
```

REPEAT / UNTIL {



```
count = 0
```

```
REPEAT
```

```
    ADD 1 to count
```

```
    WRITE count
```

```
UNTIL count < 10
```

```
WRITE "The end"
```

Mainline

```
count = 0
```

```
REPEAT
```

```
    DO Process
```

```
UNTIL count < 10
```

```
WRITE "The end"
```

Process

```
ADD 1 to count
```

```
WRITE count
```

Rules for Pseudocode {

1. Capitalize initial keywords
2. Write only one statement per line
3. Indent to show hierarchy
4. End multiline sections
5. Keep statements language independent
6. Use the naming domain of the problem

}

Capitalize Initial Keyword {

- In the example below note the words: READ and WRITE. These are just a few of the keywords to use, others include:

READ, WRITE, IF, ELSE, ENDIF, WHILE, ENDWHILE

```
READ name, hoursWorked, payrate
    gross = hoursWorked * payrate
WRITE name, hoursWorked, gross
```

}

One statement per line {

- Each statement in pseudocode should express just one action for the computer.
- If the task list is properly drawn, then in most cases each task will correspond to one line of pseudocode.

Task List

1. Read name, hours worked, rate of pay
2. Perform calculations
3. Gross = hours worked * rate of pay
4. Write name, hours worked, gross



Pseudocode

```
READ name, hoursWorked, payRate  
gross = hoursWorked * payRate  
WRITE name, hoursWorked, gross
```

Indent to Show Hierarchy {

- Each design structure uses a particular indentation pattern
- Sequence:
 - Keep statements in sequence all starting in the same column
- Selection:
 - Indent statements that fall inside selection structure, but not the keywords that form the selection

}

Indent to Show Hierarchy {

- Looping: Indent statements that fall inside the loop, but not keywords that form the loop

Task List

1. Read name, hours worked, hourly rate, deduction rate
2. Calculate gross, deduction, net pay
3. Is gross ≥ 100 ?
4. If Yes, then calculate deduction
5. If No, then no deduction
6. Write name, gross, deduction, net pay



Pseudocode

```
READ name, hoursWorked, hourlyRate,
deductionrate
grossPay = hoursWorked * hourlyRate
IF grossPay  $\geq 100$ 
    deduction = grossPay * deductionRate
ELSE
    deduction = 0
END-IF
netPay = grossPay - deduction
WRITE name, hoursWorked, gross
```

End Multiline Structures {

- See the IF/ELSE/ENDIF as constructed, the ENDIF is in line with the IF.
- The same applies for WHILE/ENDWHILE etc.

```
READ name, grossPay, taxes
```

```
IF taxes > 0
```

```
    net = grossPay - taxes
```

```
ELSE
```

```
    net = grossPay
```

```
END-IF
```

```
WRITE name, net
```

```
}
```

Language Independence {

- Resist the urge to write in whatever language you are most comfortable with; in the long run you will save time.
- Remember you are describing a logic plan to develop a program; you are not programming!



Six Basic Computer Operations {

1. A computer can receive information
2. A computer can output information
3. A computer can perform arithmetic operations
4. A computer can assign a value to a variable
5. A computer can compare two variables and select one of two alternate actions
6. A computer can repeat a group of actions

}

1.Computer can receive information {

Verb used:

- **READ** used when the algorithm is to receive the input from a record on a file
- **GET / INPUT** used when the algorithm is to receive input from the keyboard.

READ studentName

GET systemDate

READ number1, number2

GET taxCode

INPUT marks

2.Computer can output information {

Verb used:

- **PRINT** used when the output is to be sent to the printer.
- **WRITE** used when the output is to be written to a file.
- **PUT, OUTPUT, DISPLAY** used when the output is to be written to the screen.
- **PROMPT** required before an input instruction Get, causes the message to be sent to the screen which requires the user responds, usually by providing input.

}

2.Computer can output information {

```
PRINT 'Program Completed'
WRITE customer_record to master file
PUT name, address and postcode
OUTPUT total_tax
DISPLAY 'End of Data'
PROMPT for student_mark
GET student_mark
```

```
}
```

3. Computer can perform arithmetic operations {

Symbols used:

+	Add
-	Subtract
*	Multiply
/	Divide
()	Parentheses

Verb used:

- Compute
- Calculate

ADD number to total

Total = total + number

DIVIDE total_marks by student_count

Sales_tax = cost_price * 0.10

COMPUTE C = (F - 32) * 5 / 9

4.Computer can assign a value to a variable {

Three cases :

- To give data an initial value in pseudocode, the verbs INITIALIZE or SET are used.
- To assign a value as a result of some processing, the symbols '=' or '<-' are written.
- To keep a variable for later use, the verbs SAVE or STORE are used.

}

4.Computer can assign a value to a variable {

INITIALIZE total_price to zero

SET student_count to 0

total_price = cost_price + sales_tax

total_price <- cost_price + sales_tax

STORE customer_num in last_customer_num

}

5. Computer can compare two variables and select one of two alternate actions {

- To represent this operation in Pseudocode, special keywords are used: **IF**, **THEN** and **ELSE**.

```
IF student is part_time THEN
    ADD 1 to part_time count
ELSE
    ADD 1 to full_time count
END-IF
```

```
}
```

6. Computer can repeat a group of actions {

Keyword used:

- DO-WHILE, END-DO
- WHILE, END-WHILE

DO

READ 1 to part_time count

PRINT studentName, address to report

ADD 1 to student_total

WHILE student_total < 50

}

Why Use Pseudocode? {

- Pseudocode is easier to read.
- Pseudocode simplifies code construction.
- Pseudocode is a good middle point between flowchart and code.
- Pseudocode is a helpful starting point for documentation.
- Pseudocode allows for quick bug detection.

}

Rules for Variable Names {

- Begin with lowercase letter
- Contain no spaces
- Additional words begin with capital
- Unique names within code
- Consistent use of names

INITIALIZE number

SET firstName to "Kamal"

INITIALIZE offer_rate

}

Desk Checking a Pseudocode {

- Choose valid simple input test case (2-3 enough)
- Establish what the expected result should be.
- Make a table of relevant variable names
- Checking the test case line by line, step by step
- Repeat process for other test case
- Check if expected result matches with actual result

}

Advantages of Pseudocode {

- **Easy to write** - Writing of pseudocode involves much less time and effort than drawing an equivalent flowchart.
- **Easy to convert** - Converting a pseudo code to a programming language is much more easier as compared to converting a flowchart.
- **Simple to change** - It is easier to modify the pseudocode of a program logic when program modifications are necessary.

}

Disadvantages of Pseudocode {

- In case of pseudo code, a graphic representation of program logic is not available.
- There are no standard rules to follow in using pseudocode.
- Different programmers use their own style of writing pseudocode.
- Communication problems occur due to lack of standardization.
- For a beginner, it is more difficult to follow the logic or write the pseudo code, as compared to flowcharting.

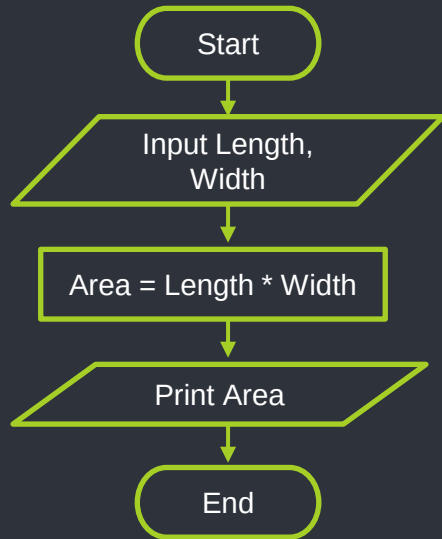
}

Algorithm vs Pseudocode {

Algorithm	Pseudocode
An Algorithm is used to provide a solution to a particular problem in form of a well-defined step-based form.	A Pseudocode is a step-by-step description of an algorithm in code-like structure using plain English text.
An algorithm only uses simple English words	Pseudocode also uses reserved keywords like if-else, for, while, etc.
These are a sequence of steps of a solution to a problem	These are fake codes as the word pseudo means fake, using code like structure and plain English text
There are no rules to writing algorithms	There are certain rules for writing pseudocode
Algorithms can be considered pseudocode	Pseudocode cannot be considered an algorithm
It is difficult to understand and interpret	It is easy to understand and interpret

Examples of Pseudocode {

- Find the area of rectangle



BEGIN

READ Length, Width

Area = Length * Width

DISPLAY Area

END

Examples of Pseudocode {

- Pseudocode to find the maximum number in a list of numbers.

BEGIN

INPUT list of numbers

SET max to first number in the list

FOR each number in the list

IF number > max **THEN**

SET max to number

END-IF

END-FOR

OUTPUT max

END

}

Examples of Pseudocode {

- Pseudocode that will take two numbers as input, calculates their sum and displays output.

BEGIN

WRITE "Please enter two numbers to add"

READ num1

READ num2

Sum = num1 + num2

WRITE "The Answer is: ", Sum

END

}

Examples of Pseudocode {

- Pseudocode that will take marks of physics, chemistry and math as input, calculates the average and displays output.

BEGIN

WRITE "please enter marks of physics"

INPUT Phy_marks

WRITE "please enter marks of chemistry"

INPUT Chem_marks

WRITE "please enter marks of maths"

INPUT math_marks

Avg = (Phy_marks + Chem_marks + math_marks)/3

DISPLAY Avg

END

}

{ Thank You }